



DURBAN UNIVERSITY OF TECHNOLOGY
INYUVESI YASETHEKWINI YEZOBUCHWEPHESHE

FACULTY OF ACCOUNTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

2023

MID YEAR MAIN EXAMINATION

INSTRUCTIONAL PROGRAMME : B INFORMATION & COMMUNICATION TECHNOLOGY

INSTRUCTIONAL OFFERING : INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

PAPER NUMBER : 1

SUBJECT CODE/S : IPRT301

DATE : 26 JUNE 2023

DURATION : 3 HOURS

TIME : 14H00 - 17H00

TOTAL MARKS : 100

NUMBER OF PAGES : 7 (INCLUDING COVER PAGE)

EXAMINER/S : BOAMAH-ABU, CHARLES (PhD)

MODERATOR/S : IBIDUN OBAGBUWA, CHRISTIANA (PhD)

INSTRUCTIONS/REQUIREMENTS:-

1. This examination is closed book
2. Answer all questions
3. Where stated "briefly", please answer in a few lines
4. Ensure your handwriting is readable

Do not turn the page until permission is given

Question 1

10 marks

In each of the following, indicate whether the statement is **true** or **false**.

1. In Integrative programming, at least one of the components to be integrated must exist.
2. An abstract class can extend another abstract class.
3. You must convert a class which contains variables and method implementations to an interface.
4. The main purpose of an abstract classes is code reuse.
5. Any class that extends an abstract class must also be declared abstract.
6. Dynamic binding is linked to method override.
7. Design patterns determine what programming languages to use.
8. Organizations can develop their own design patterns.
9. RMI allows the use of different programming languages for the server and client applications.
10. Server-side scripts are faster to process, compared to client-side scripts.

Question 2

4 marks

Match the following scenarios in software design to the appropriate design patterns. **Write only the number and the corresponding letter.**

	Scenario		Design pattern
1	Separating business objects from operations	A	Singleton
2	Creating instances of subclasses	B	Adaptive
3	Allowing different classes to work together	C	DAO
4	Creating only one instance of a class	D	MVC
		E	Factory

Question 3

3 marks

Match the following descriptions to the appropriate distributed architecture. **Write only the number and the corresponding letter.**

	Description		Architecture
1	Uses XML for data exchange	A	Microservice
2	Allows invocation of the methods of remote objects	B	Client-server
3	Consists of small independent services, with their own processor	C	Web service
		D	Broker

Question 4

12 marks

1. "Integrative programming improves interoperability of heterogeneous hardware and software resources". Briefly explain. **(2 marks)**
2. State and briefly describe one advantage of an interface over abstract class. **(2 marks)**
3. "Interfaces or abstracts classes are preferred to concrete class". Explain why. **(2 marks)**
4. Briefly explain the importance of standardization in data exchange. Illustrate your answer with an example. **(4 marks)**
5. Explain why multithreading is an important feature of distributed applications. **(2 marks)**

Question 5

15 marks

The following shows part of an XML document (**fees.xml**). Each **Student** node contains an **id** (a unique identifier), **name** (of the student) and **num** (number of modules registered).

```
<?xml version="1.0" encoding="UTF-8"?>
<Account>
  <Student id="20211">
    <name>L Lucas</name>
    <num>3</num>
  </Student>
  <Student id="20212">
    <name>M Awuah</name>
    <num>5</num>
  </Student>
  ...
</Account>
```

1. Complete the DTD file (**fees.dtd**) below. (6 marks)

```
<?xml version="1.0" encoding="UTF-8"?>
<!ELEMENT Account (Student)*>
_____
_____
_____
_____
```

2. Complete the tag below which links **fees.xml** to the structure in **fees.dtd**. (3 marks)

```
<!DOCTYPE _____>
```

3. Assume the fee per module is R8000, complete the XSL file (**fees.xsl**) below to displays the **id**, **name** and **num** (from **fees.xml**) and the total **fees** for each student. (5 marks)

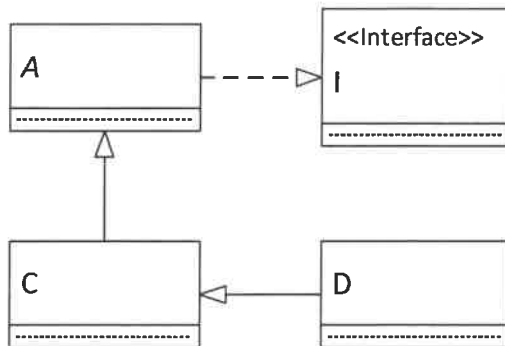
Student Id	Name	No. of modules	Fees
20211	J Lucas	3	24000
20212	M Awuah	5	40000

```
<xsl:template match="/Account">
  .....
  <xsl:for-each select="_____">
    <tr>
      <td>_____</td>
      <td>_____</td>
      <td>_____</td>
      <td>_____</td>
    </tr>
  </xsl:for-each>
  .....
```


Question 6

10 marks

The figure below shows **I** (an interface), **A** (an abstract class), **C** (a concrete class) and **D** (a concrete class).



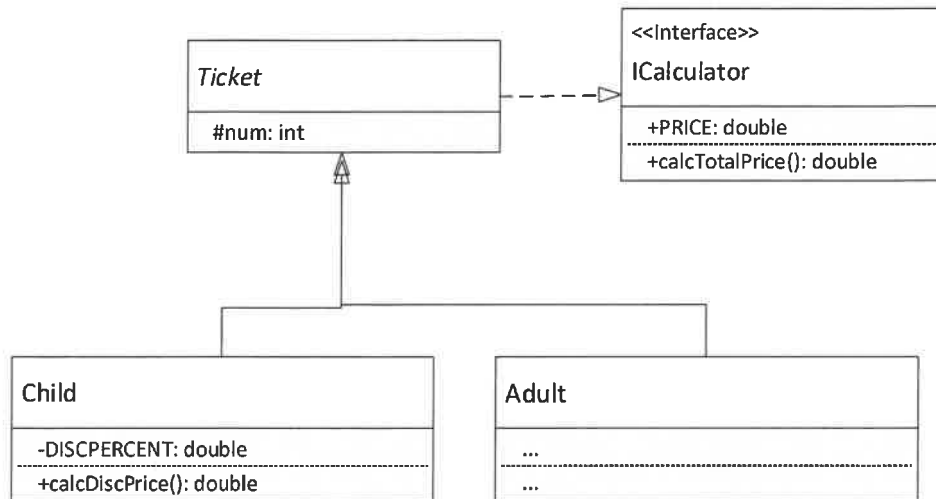
In each of the following, state whether the statement is **valid** or **invalid**. Explain your answer

1. `I ia=new I();`
2. `I ib=new A();`
3. `I ic=new C();`
4. `A aa=new C();`
5. `D dd=new C();`

Question 7

10 marks

NLM sells two types of concerts tickets: **child** and **adult**. There is 20% discount on **child** tickets. The figure below shows the classes (**Ticket**, **Child** and **Adult**) and an interface (**ICalculator**). The method **calcTotalPrice()** returns the total price (based on the number of ticket bought). **PRICE** per ticket is R100 (a constant) and **num** is the number of tickets bought. **DISCPERCENT** is the percentage discount (a constant, 20%). The method **calcDiscPrice()** in **Child** returns the discounted price, i.e., total price less discount (note: **calcDiscPrice()** calls **calcTotalPrice()**).



A typical output for **Child** ticket is shown below:

```

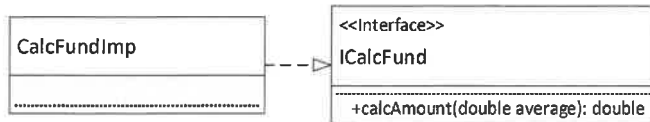
No. of ticket=10
Total Price=R1000.0
Discounted price=R800.0
  
```

1. Write the method **calcTotalPrice()** in **ICalculator**. (2 marks)
2. Write the class header for **Ticket**. (2 marks)
3. Write the method **calcDiscPrice()** in **Child**. (4 marks)
4. Using the classes **Ticket** and **Child**, briefly explain the term "polymorphism". (2 marks)

Question 8

17 marks

You are required to create an RMI application. The figure below shows **ICalcFund** (the remote interface). The class **CalcFundImp** implements **ICalcFund**. The method **calcAmount(double average)** takes the average mark of a student as parameter and returns the amount to allocate.



1. Write the header for **ICalcFund**. (2 marks)
2. Write the method **calcAmount(double average)** in **ICalcFund**. (3 marks)
3. Write the class header for **CalcFundImp**. (2 marks)
4. Complete the class **Server** (below) which registers and binds an instance of the remote object. Name the remote object **fund**; use the port **1990**. (5 marks)

```

public class Server {
    public static void main(String[] args) _____ {
        CalcFundImp cf=new CalcFundImp();//Create instance
        Registry reg=_____
        _____
    }
}
  
```

5. Complete the class **Client** (below) which calls **calcAmount(double average)** of the remote object with the value 80.0 and displays the result on the screen. Use the local host. (5 marks)

```

public class Client {
    public static void main(String[] args)_____ {
        _____ ic=_____
        System.out.println("The amount is "+ _____)
    }
}
  
```

Question 9

12 marks

Complete the code below which:

1. Connects to a URL resource (a file "**fees.dtd**" in the folder **webapp** on a **local host**) on port **8080**
2. Reads the content of the resource and
3. Display on the screen.

Use **openStream()**. Also, the program must close all streams.

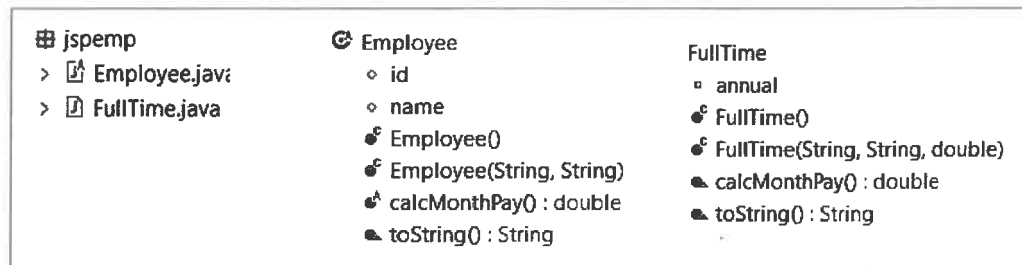
```

public void read() throws Exception{
    _____
    _____
    _____
    _____
    _____
}
  
```

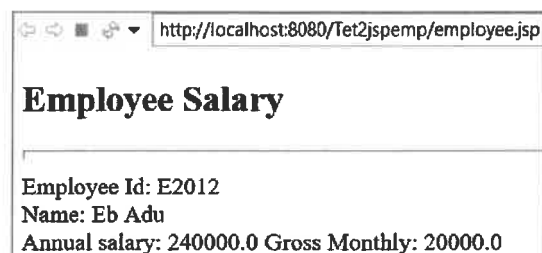

Question 10

8 marks

The figure below shows the package **jspemp** contains classes **Employee** (abstract) and **FullTime** (which extends **Employee**). The method **toString()** returns the string representation of an object of the class.



The package is imported into JSP code to display an object of an employee in a web page (as shown below).



Assume the classes in package **jspemp** have been implemented.

1. Complete the JSP page directive (below), which specifies the programming language imports the classes **Employee** and **FullTime**. (4 marks)

<%@ page language="_____" import="_____" %>

2. Write the JSP scriptlet, which creates an instance of **FullTime** and displays the instance in the web page. (4 marks)

